

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

по дисциплине «Системное программирование»

Выполнил:

Студент группы Р4114

Рыбин В. А.

Санкт-Петербург

2025

Задание

Реализовать формирование линейного кода в терминах некоторого набора инструкций посредством анализа графа потока управления для набора подпрограмм. Полученный линейный код вывести в мнемонической форме в выходной текстовый файл.

Подготовка к выполнению по одному из двух сценариев:

1 Составить описание виртуальной машины с набором инструкций и моделью памяти по варианту

a. Изучить нотацию для записи определений целевых архитектур

b. Составить описание ВМ в соответствии с вариантом

i. Описание набор регистров и банков памяти

ii. Описать набор инструкций: для каждой инструкции задать структуру операционного кода, содержащего описание операндов и набор операций, изменяющих состояние ВМ

1 Описать инструкции перемещения данных и загрузки констант

2 Описать инструкции арифметических и логических операций

3 Описать инструкции условной и безусловной передачи управления

4 Описать инструкции ввода-вывода с использованием скрытого регистра в качестве порта ввода-вывода

iii. Описать набор мнемоник, соответствующих инструкциям ВМ

c.

Подготовить скрипт для запуска ассемблированного листинга с использованием описания ВМ:

i. Написать тестовый листинг с использованием подготовленных мнемоник инструкций

ii. Задействовать транслятор листинга в бинарный модуль по описанию ВМ

iii. Запустить полученный бинарный модуль на исполнение и получить результат работы

iv. Убедиться в корректности функционирования всех инструкций ВМ

2 Выбрать и изучить прикладную архитектуру системы команд существующей ВМ

a. Для выбранной ВМ:

i. Должен существовать готовый эмулятор (например qemu)

ii. Должен существовать готовый тулчейн (набор инструментов разработчика): компилятор Си, ассемблер и дизассемблер, линковщик, желательно отладчик

b. Согласовать выбор ВМ с преподавателем

c.

Изучить модель памяти и набор инструкций ВМ

d. Научиться использовать тулчейн (собирать и запускать программы из листинга)

e. Подготовить скрипт для запуска ассемблированного листинга с использованием эмулятора

i. Написать тестовый листинг с использованием инструкций ВМ

ii. Задействовать ассемблер и компоновщик из тулчейна

iii. Запустить бинарный модуль на исполнение и получить результат его работы

Порядок выполнения:

1 Описать структуры данных, необходимые для представления информации об элементах образа программы (последовательностях инструкций и данных), расположенных в памяти

- a. Для каждой инструкции – имя мнемоники и набор операндов в терминах данной VM
- b. Для элемента данных – соответствующее литеральное значение или размер экземпляра типа данных в байтах

2 Реализовать модуль, формирующий образ программы в линейном коде для данного набора подпрограмм

- a. Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах для набора подпрограмм, разработанную в задании 2 (п. 1.a, п. 2.b)
- b. В результате работы порождается структура данных, разработанная в п. 1, содержащая описание образа программы в памяти: набор именованных элементов данных и набор именованных фрагментов линейного кода, представляющих собой алгоритмы подпрограмм
- c.

Для каждой подпрограммы посредством обхода узлов графа потока управления в порядке топологической сортировки (начиная с узла, являющегося первым базовым блоком алгоритма подпрограммы), сформировать набор именованных групп инструкций, включая пролог и эпилог подпрограммы (формирующие и разрушающие локальное состояние подпрограммы)

- d. Для каждого базового блока в составе графа потока управления сформировать группу инструкций, соответствующих операциям в составе дерева операций
- e. Использовать имена групп инструкций для формирования инструкций перехода между блоками инструкций, соответствующих узлам графа потока управления, в соответствии с дугами в нём

3 Доработать тестовую программу, разработанную в задании 2 для демонстрации работоспособности созданного модуля

- a. Добавить поддержку аргумента командной строки для имени выходного файла, вывод информации о графах потока управления сделать опциональных
- b. Использовать модуль, разработанный в п. 2 для формирования образа программы на основе информации, собранной в результате работы модуля, созданного в задании 2 (п. 2.b)
- c. Для сформированного образа программы в линейном коде вывести в выходной файл

ассемблерный листинг, содержащий мнемоническое представление инструкций и данных, как
они описаны в структурах данных (п. 1), построенных разработанным модулем (пп. 2.с-е)
d. Проверить корректность решения посредством сборки сгенерированного листинга и запуска
полученного бинарного модуля на эмуляторе ВМ (см. подготовка п. 1.с или п. 2.е)
4 Результаты тестирования представить в виде отчета, в который включить:
a. В части 3 привести описание разработанных структур данных
b. В части 4 описать программный интерфейс и особенности реализации
разработанного модуля
с.
В части 5 привести примеры исходных текстов, соответствующие ассемблерные листинги и
примера вывода запущенных тестовых программ

Структуры данных

```
typedef struct
{
    OperandType operand_type;

    Register reg;
    int value;
    Stack stack;
    char lable[LABEL_LENGTH];
} Operand;

typedef struct
{
    Mnemonic mnemonic;
    size_t operand_amount;
    Operand first_operand;
    Operand second_operand;
    Operand third_operand;
} Instruction;

typedef struct
{
    char buffer[LABEL_LENGTH];
} Label;
```

```
typedef struct
{
    bool is_label;
    bool is_empty;
    union
    {
        Instruction instruction;
        Label label;
    } data;
} Line;
```

Примеры

Код:

```
function main()
    dim result, a, b as int
    result = a + b;
end function
```

Asm:

```
.section .data

.section .text
.global main
main:
    addi sp, sp, -96
    sd ra, 8(sp)
    sd s0, 0(sp)
    addi s0, sp, 96
    lw t0, -20(s0)
    lw t1, -24(s0)
    add t0, t0, t1
    sw t0, -16(s0)
    ld ra, 8(sp)
    ld s0, 0(sp)
    addi sp, sp, 96
    ret
```

Код:

```
function foo(a as int, b as int):
    dim result as int
    result = a * 2 + b;
    result;
end function
```

```
foo:
    addi sp, sp, -96
    sd ra, 8(sp)
    sd s0, 0(sp)
    addi s0, sp, 96
    sw a0, -16(s0)
    sw a1, -20(s0)
    lw t0, -16(s0)
    addi t1, zero, 2
    mul t0, t0, t1
    lw t1, -20(s0)
    add t0, t0, t1
    sw t0, -24(s0)
    lw t0, -24(s0)
    mv a0, t0
    ld ra, 8(sp)
    ld s0, 0(sp)
    addi sp, sp, 96
    ret
```

Вывод:

Были изучены и освоены на практике основы генерации asm кода под платформу RISC-V.